UNIVERSITY OF SZEGED
Department of Software Engineering
UNIVERSITAS SCIENTIARUM SZEGEDIENSIS

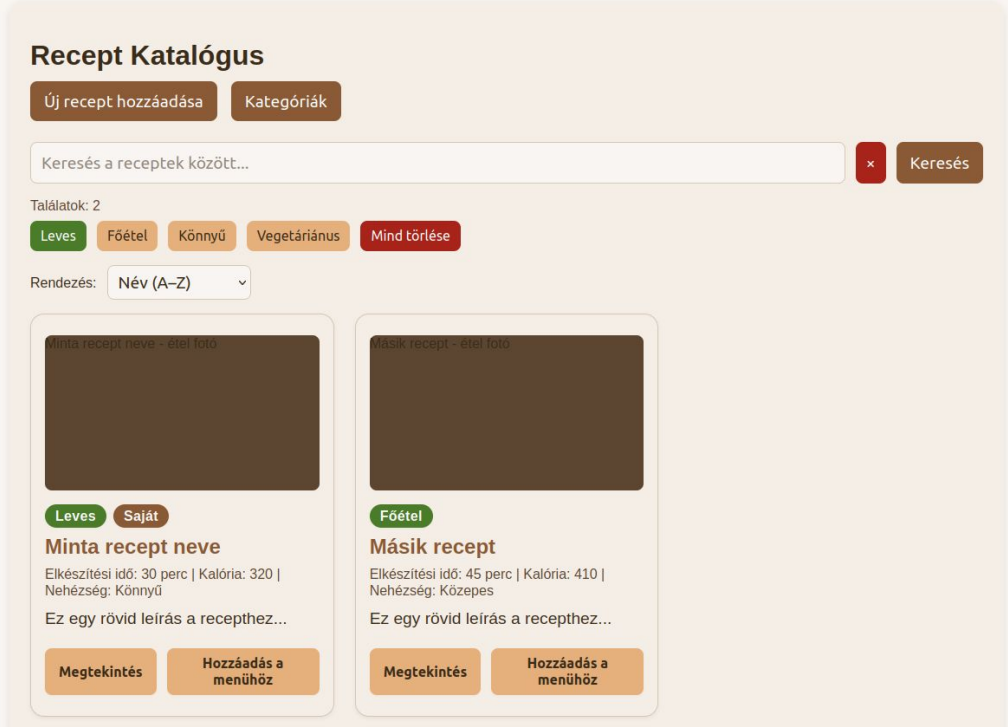
Webfejlesztési keretrendszerek

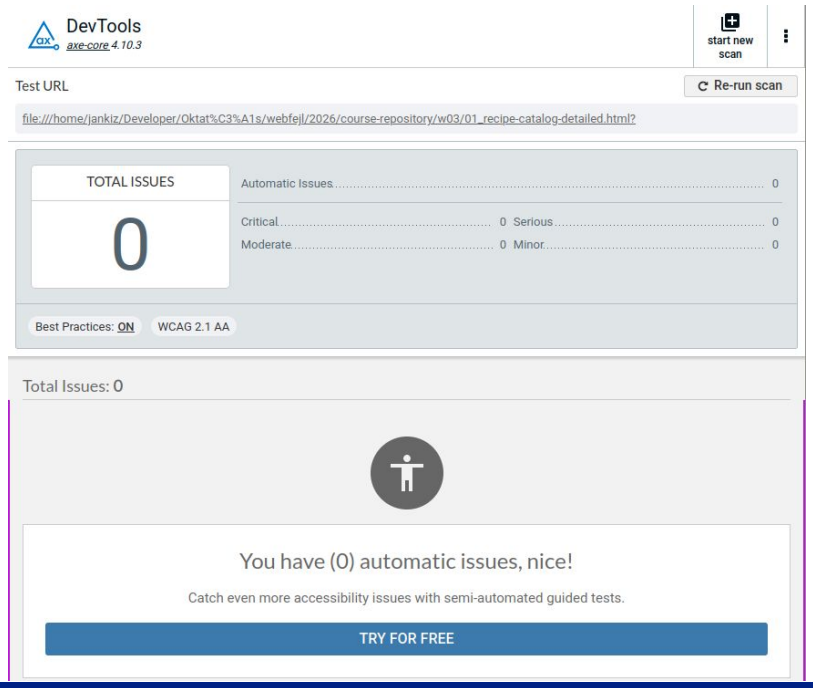
4. gyakorlat

Dr. Jánki Zoltán Richárd

Hol tartunk?

- Specifikáció → kész!
- UI tervek és design token rendszer→ kész!
- Akadálymentességi tervek → kész





Tisztán HTML?

- Egyetlen HTML?
  - kezelhetetlen, átláthatatlan
  - egyetlen igény → sok manuális változtatás
  - csapatmunka nem lehetséges
  - újrafelhasználhatóság nincs
- Logika?
  - JavaScript szükséges

Ötlet: bontsuk a rendszert komponensekre!

Platform, keretrendszer

- Keretrendszer
  - meghatározott struktúra, szabályok
  - eszközkészlet a fejlesztőknek
  - amit nyújt:
    - komponens-modell
    - reaktivitás
    - ökoszisztéma
  - minden olyan dolog, amit natív HTML/CSS/JS-sel szintén meg lehet oldani

Népszerű keretrendszerek

- Angular
  - teljes keretrendszer
  - nagyobb tanulási görbe
  - kötöttebb módszerek
- React
  - UI könyvtár (JSX-alapú)
  - nagyobb szabadságfok
  - legnagyobb ökoszisztéma és közösség
- Vue
  - könnyen tanulható progresszív keretrendszer
  - “arany középút”
- Svelte
  - fordítási időben dolgozik
  - komponensekből VanillaJS

Közös: mind komponens-alapú, mind reaktív, az elvek mindre érvényesek

UNIVERSITY OF SZEGED
Department of Software Engineering
UNIVERSITAS SCIENTIARUM SZEGEDIENSIS

Komponens fogalma

- A UI egy önálló újrafelhasználható építőeleme
  - Sablon (template): HTML
  - Logika: JavaScript
  - Stílus: CSS
- Az alkalmazások fa-struktúrát alkotnak
- Alapelvek
  - Egyetlen felelősség
  - Újrafelhasználhatóság
  - Kis méret
  - Egyértelmű interfész
  - Buta vs. okos komponensek

```
App
├── Header
│   ├── Logo
│   └── Navigation
├── Main Content
│   ├── SearchBar
│   ├── FilterPanel
│   └── CardGrid
│       ├── Card
│       ├── Card
│       └── Card
└── Footer
```

Komponensek közötti kommunikáció

1. Szülő→ gyerek
2. Gyerek → szülő
3. Távoli komponensek között

Legjobb gyakorlat: egyirányú adatáramlás (felfelé vagy lefelé)

Anti-pattern:

- azonnali globális store használat
- adatátadás 4-5 szinten keresztül
- kétirányú kötés túlzott használata

Komponens életciklus (Lifecycle)

1. Létrehozás
2. Változás-detektálás
3. Frissítés
4. Megsemmisülés

Fontosabb hook-ok (minden keretrendszerben):
- Inicializálás → adatok betöltése
- Bemenet változása → változás figyelése
- Megsemmisülés → cleanup (takarítás)

Tipikus életciklushibák

- Hiányzó cleanup → memory leak
- Rossz hook választás
- Túl gyakori változás detektálás → felesleges renderelés
- Aszinkron műveletek versenye

Komponensekre bontás a gyakorlatban

- 3 fő kérdés
  - Megjelenik-e az elem az alkalmazás több pontján is?
    - Ha igen, akkor önálló komponens.
  - Van-e önálló felelősségi köre?
    - Ha igen, válasszuk le!
  - Kezelhetetlenül nagy fájl lett?
    - Ha 300 sornál hosszabb a kód, valószínűleg tovább bontható.

Gyakorlat

- Bontsuk fel az alkalmazásunkat komponensekre!
  - Használjuk a specifikációt (meghatározzuk a felelősségeket)
  - Használjuk a UI terveket (lássuk, mi az, ami többször megjelenik)
- Valósítsuk meg a komponenseket!

Tippek

- AI használat tipp: ai-digest csomag összegyűjti a teljes kódbázist
- Ha hiba van, adjuk át a hibaüzenetet.
  - Ha fordítási időben jött elő, akkor más nem kell
  - Ha futási időben jött elő, mondjuk meg, hogy minek hatására
- Legjobb debug eszköz: F12 (Inspect)
  - A console eszközön keresztül kiolvasható a hiba
  - Az inspect-en keresztül kipróbálhatóak a UI módosítások